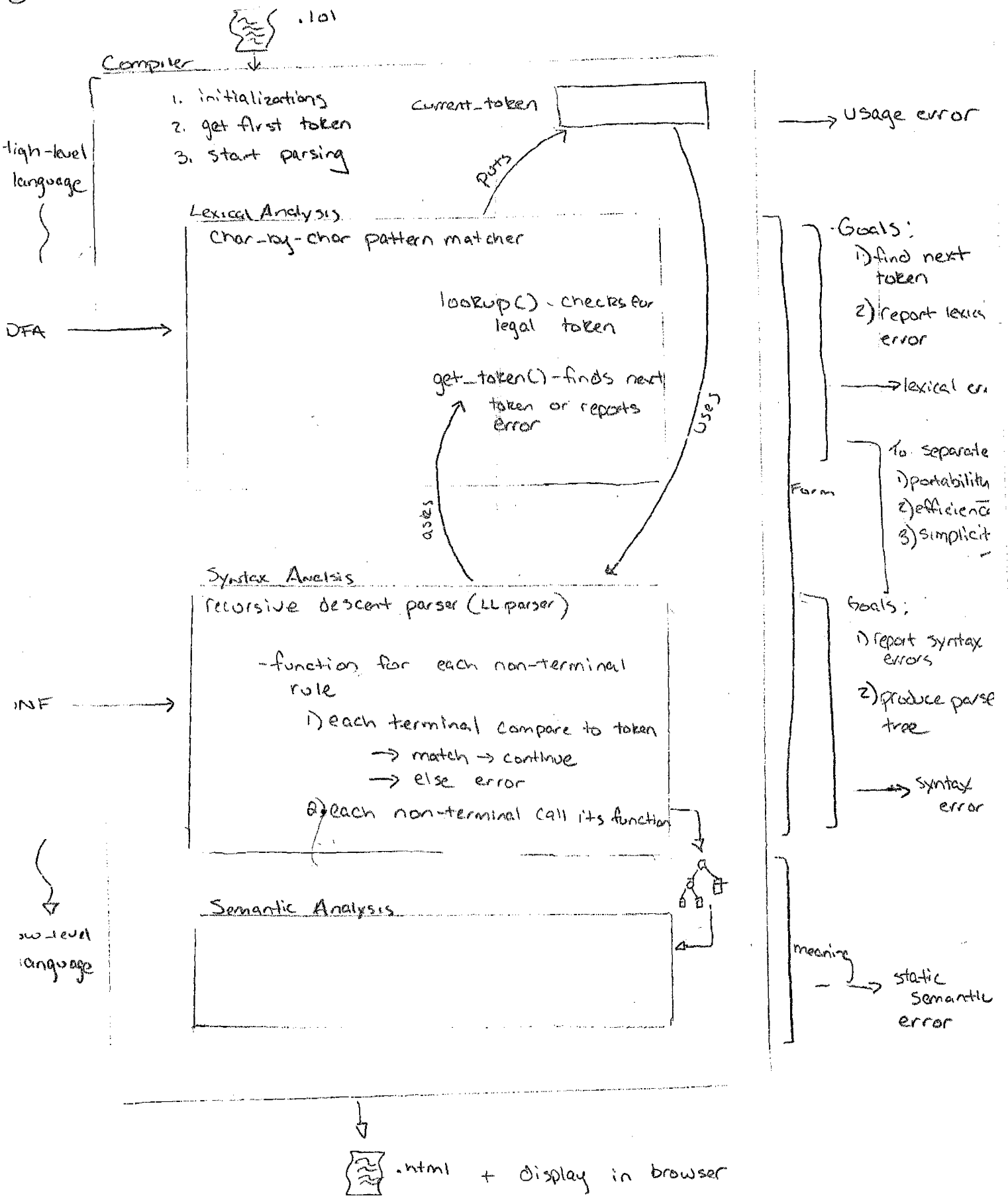
(6)

DETAILED


.101
Compiler
1. initializations
2. get first token
3. start parsing
high-level language
current_token
puts
Lexical Analysis
Char-by-char pattern matcher
lookup() - checks for legal token
get_token() - finds next token or reports error
DFA
uses
asks
Syntax Analysis
recursive descent parser (LL parser)
-function for each non-terminal rule
1) each terminal compare to token
→ match → continue
→ else error
2) each non-terminal call its function
BNF
Semantic Analysis
low-level language
usage error
Goals:
1) find next token
2) report lexical error
lexical er
Form
To separate
1) portability
2) efficiency
3) simplicity
Goals:
1) report syntax errors
2) produce parse tree
syntax error
meaning
static semantic error
.html + display in browser

Compiler

→ LAB 5 solution (I'll post my solution)

>lolcompiler.exe input.lol

high level ⟶ low-level

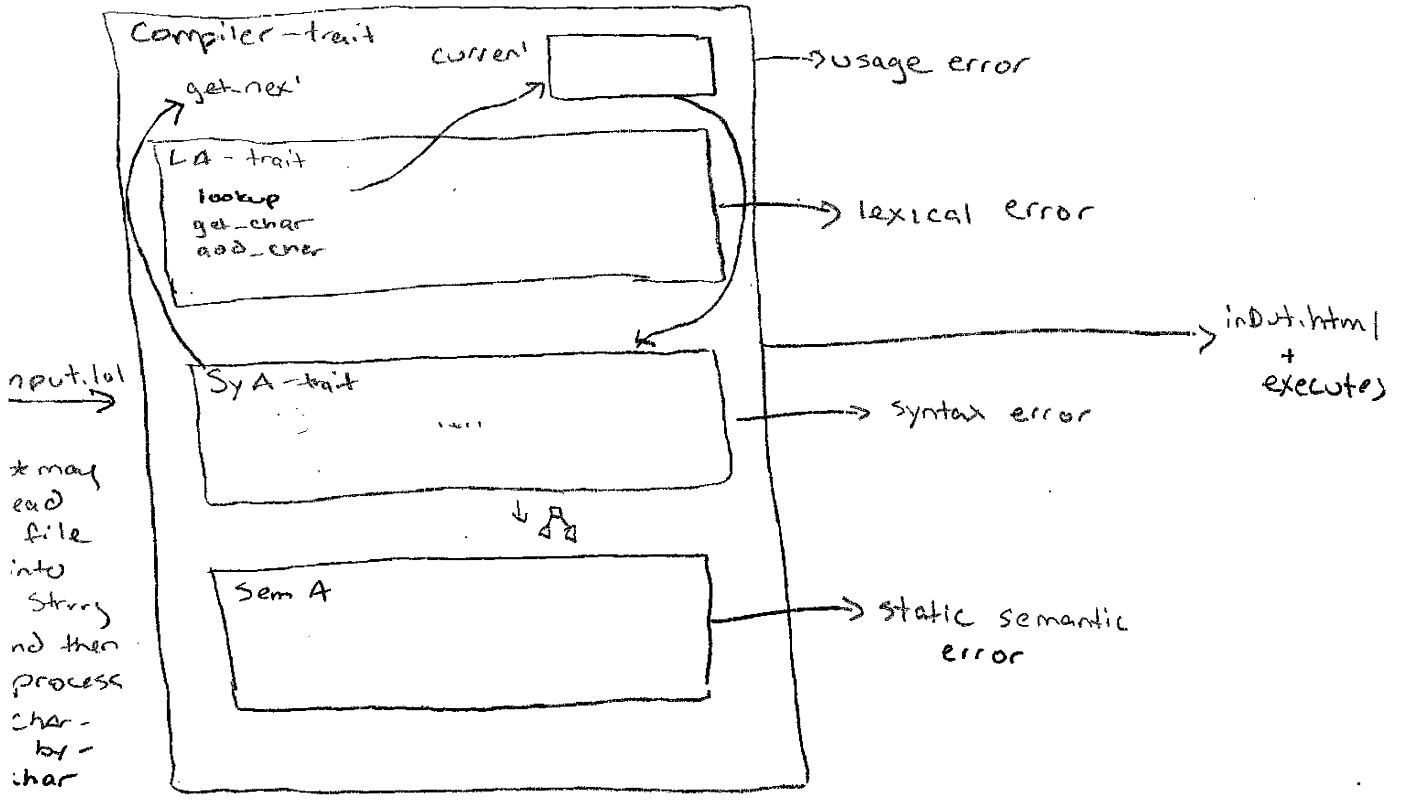
Success:

1) produces input.html
2) executes by opening input.html in chrome

Failure:

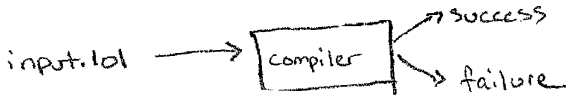
1) usage error
2) lexical error
3) syntax error
4) static semantic error

If any error, no output file but console, helpful error msg and quit.



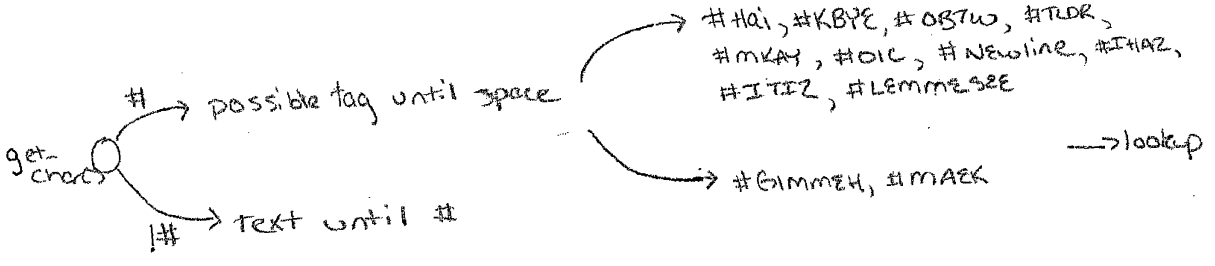
①

>lolcompiler.exe input.lol



③ Lexical Analysis

when get_next_token() is called:

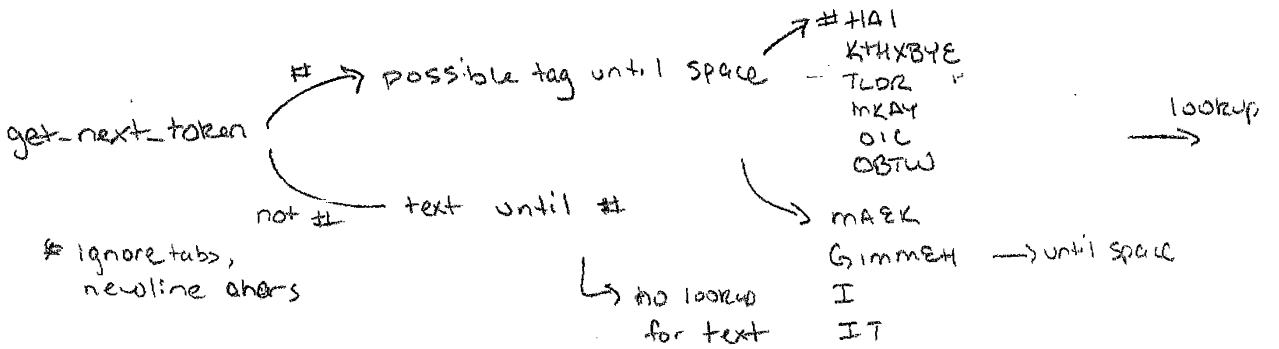


④ Syntax Analysis

- Use BNF grammar to develop parsing methods
- If token is legal, put on a vector/stack/"parse tree", get_next_token, move on

Lexical Analysis

Finite State Machine



if text or lookup says valid, put in current token
else error

Syntax Analysis – lab 5

use grammar to develop pasing methods

if token is legal, put on a vector/array/stack, get_token move on
else error

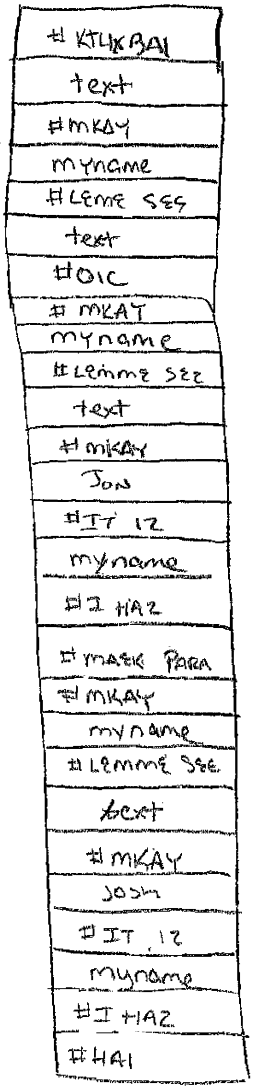
parse tree

Semantic Analysis

- resolve vars (next week)
- go through parse tree and resolve to html tag
- output to file and execute

STATIC SCOPE

Parse → tree



TYPES OF TOKENS

1) Begin block tags → push onto stack
2) text → push onto stack
3) newline → push onto stack
4) VARIABLE DEF → get name and value, "push" onto stack
5) VARIABLE USE → from top of stack, search for variable
   if found, resolve with value
   else, static semantic error + quit
6) END BLOCK tags → from top of stack, traverse/pop to remove and VARIBLE DEF UNTIL CORRESPONDING BEGIN TAG